

用来替代 `s` 的内容。

11.3.6 节练习

练习 11.33: 实现你自己版本的单词转换程序。

练习 11.34: 如果你将 `transform` 函数中的 `find` 替换为下标运算符, 会发生什么情况?

练习 11.35: 在 `buildMap` 中, 如果进行如下改写, 会有什么效果?

```
trans_map[key] = value.substr(1);
改为 trans_map.insert({key, value.substr(1)})
```

练习 11.36: 我们的程序并没有检查输入文件的合法性。特别是, 它假定转换规则文件中的规则都是有意义的。如果文件中的某一行包含一个关键字、一个空格, 然后就结束了, 会发生什么? 预测程序的行为并进行验证, 再与你的程序进行比较。



11.4 无序容器

新标准定义了 4 个无序关联容器 (unordered associative container)。这些容器不是使用比较运算符来组织元素, 而是使用一个哈希函数 (hash function) 和关键字类型的 `==` 运算符。在关键字类型的元素没有明显的序关系的情况下, 无序容器是非常有用的。在某些应用中, 维护元素的序代价非常高昂, 此时无序容器也很有用。

虽然理论上哈希技术能获得更好的平均性能, 但在实际中想要达到很好的效果还需要进行一些性能测试和调优工作。因此, 使用无序容器通常更为简单 (通常也会有更好的性能)。

444



如果关键字类型固有就是无序的, 或者性能测试发现问题可以用哈希技术解决, 就可以使用无序容器。

使用无序容器

除了哈希管理操作之外, 无序容器还提供了与有序容器相同的操作 (`find`、`insert` 等)。这意味着我们曾用于 `map` 和 `set` 的操作也能用于 `unordered_map` 和 `unordered_set`。类似的, 无序容器也有允许重复关键字的版本。

因此, 通常可以用一个无序容器替换对应的有序容器, 反之亦然。但是, 由于元素未按顺序存储, 一个使用无序容器的程序的输出 (通常) 会与使用有序容器的版本不同。

例如, 可以用 `unordered_map` 重写最初的单词计数程序 (参见 11.1 节, 第 375 页):

```
// 统计出现次数, 但单词不会按字典序排列
unordered_map<string, size_t> word_count;
string word;
while (cin >> word)
    ++word_count[word];           // 提取并递增 word 的计数器
for (const auto &w : word_count) // 对 map 中的每个元素
    // 打印结果
    cout << w.first << " occurs " << w.second
        << ((w.second > 1) ? " times" : " time") << endl;
```

此程序与原程序的唯一区别是 `word_count` 的类型。如果在相同的输入数据上运行此版本，会得到这样的输出：

```
containers. occurs 1 time
use occurs 1 time
can occurs 1 time
examples occurs 1 time
...
```

对于每个单词，我们将得到相同的计数结果。但单词不太可能按字典序输出。

管理桶

无序容器在存储上组织为一组桶，每个桶保存零个或多个元素。无序容器使用一个哈希函数将元素映射到桶。为了访问一个元素，容器首先计算元素的哈希值，它指出应该搜索哪个桶。容器将具有一个特定哈希值的所有元素都保存在相同的桶中。如果容器允许重复关键字，所有具有相同关键字的元素也都会在同一桶中。因此，无序容器的性能依赖于哈希函数的质量和桶的数量和大小。

对于相同的参数，哈希函数必须总是产生相同的结果。理想情况下，哈希函数还能将每个特定的值映射到唯一的桶。但是，将不同关键字的元素映射到相同的桶也是允许的。当一个桶保存多个元素时，需要顺序搜索这些元素来查找我们想要的那个。计算一个元素的哈希值和桶中搜索通常都是很快的操作。但是，如果一个桶中保存了很多元素，那么查找一个特定元素就需要大量比较操作。

◀ 445

无序容器提供了一组管理桶的函数，如表 11.8 所示。这些成员函数允许我们查询容器的状态以及在必要时强制容器进行重组。

表 11.8：无序容器管理操作	
桶接口	
<code>c.bucket_count()</code>	正在使用的桶的数目
<code>c.max_bucket_count()</code>	容器能容纳的最多的桶的数量
<code>c.bucket_size(n)</code>	第 <code>n</code> 个桶中有多少个元素
<code>c.bucket(k)</code>	关键字为 <code>k</code> 的元素在哪个桶中
桶迭代	
<code>local_iterator</code>	可以用来访问桶中元素的迭代器类型
<code>const_local_iterator</code>	桶迭代器的 <code>const</code> 版本
<code>c.begin(n), c.end(n)</code>	桶 <code>n</code> 的首元素迭代器和尾后迭代器
<code>c.cbegin(n), c.cend(n)</code>	与前两个函数类似，但返回 <code>const_local_iterator</code>
哈希策略	
<code>c.load_factor()</code>	每个桶的平均元素数量，返回 <code>float</code> 值
<code>c.max_load_factor()</code>	<code>c</code> 试图维护的平均桶大小，返回 <code>float</code> 值。 <code>c</code> 会在需要时添加新的桶，以使得 <code>load_factor<=max_load_factor</code>
<code>c.rehash(n)</code>	重组存储，使得 <code>bucket_count>=n</code> 且 <code>bucket_count>size/max_load_factor</code>
<code>c.reserve(n)</code>	重组存储，使得 <code>c</code> 可以保存 <code>n</code> 个元素且不必 <code>rehash</code>

无序容器对关键字类型的要求

默认情况下，无序容器使用关键字类型的 `==` 运算符来比较元素，它们还使用一个 `hash<key_type>` 类型的对象来生成每个元素的哈希值。标准库为内置类型（包括指针）提供了 **hash** 模板。还为一些标准库类型，包括 `string` 和我们将来在第 12 章介绍的智能指针类型定义了 `hash`。因此，我们可以直接定义关键字是内置类型（包括指针类型）、`string` 还是智能指针类型的无序容器。

但是，我们不能直接定义关键字类型为自定义类类型的无序容器。与容器不同，不能直接使用哈希模板，而必须提供我们自己的 `hash` 模板版本。我们将在 16.5 节（第 626 页）中介绍如何做到这一点。

我们不使用默认的 `hash`，而是使用另一种方法，类似于为有序容器重载关键字类型的默认比较操作（参见 11.2.2 节，第 378 页）。为了能将 `Sale_data` 用作关键字，我们需要提供函数来替代 `==` 运算符和哈希值计算函数。我们从定义这些重载函数开始：

```
size_t hasher(const Sales_data &sd)
{
    return hash<string>()(sd.isbn());
}
bool eqOp(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() == rhs.isbn();
}
```

我们的 `hasher` 函数使用一个标准库 `hash` 类型对象来计算 ISBN 成员的哈希值，该 `hash` 类型建立在 `string` 类型之上。类似的，`eqOp` 函数通过比较 ISBN 号来比较两个 `Sales_data`。

我们使用这些函数来定义一个 `unordered_multiset`

```
using SD_multiset = unordered_multiset<Sales_data,
                                     decltype(hasher)*, decltype(eqOp)*>;
// 参数是桶大小、哈希函数指针和相等性判断运算符指针
SD_multiset bookstore(42, hasher, eqOp);
```

为了简化 `bookstore` 的定义，首先为 `unordered_multiset` 定义了一个类型别名（参见 2.5.1 节，第 60 页），此集合的哈希和相等性判断操作与 `hasher` 和 `eqOp` 函数有着相同的类型。通过使用这种类型，在定义 `bookstore` 时可以将我们希望它使用的函数的指针传递给它。

如果我们的类定义了 `==` 运算符，则可以只重载哈希函数：

```
// 使用 FooHash 生成哈希值；Foo 必须有 == 运算符
unordered_set<Foo, decltype(FooHash)*> fooSet(10, FooHash);
```

11.4 节练习

练习 11.37：一个无序容器与其有序版本相比有何优势？有序版本有何优势？

练习 11.38：用 `unordered_map` 重写单词计数程序（参见 11.1 节，第 375 页）和单词转换程序（参见 11.3.6 节，第 391 页）。

小结

447

关联容器支持通过关键字高效查找和提取元素。对关键字的使用将关联容器和顺序容器区分开来，顺序容器中是通过位置访问元素的。

标准库定义了 8 个关联容器，每个容器

- 是一个 `map` 或是一个 `set`。`map` 保存关键字-值对；`set` 只保存关键字。
- 要求关键字唯一或不要求。
- 保持关键字有序或不保证有序。

有序容器使用比较函数来比较关键字，从而将元素按顺序存储。默认情况下，比较操作是采用关键字类型的 `<` 运算符。无序容器使用关键字类型的 `==` 运算符和一个 `hash<key_type>` 类型的对象来组织元素。

允许重复关键字的容器的名字中都包含 `multi`；而使用哈希技术的容器的名字都以 `unordered` 开头。例如，`set` 是一个有序集合，其中每个关键字只可以出现一次；`unordered_multiset` 则是一个无序的关键字集合，其中关键字可以出现多次。

关联容器和顺序容器有很多共同的元素。但是，关联容器定义了一些新操作，并对一些和顺序容器和关联容器都支持的操作重新定义了含义或返回类型。操作的不同反映出关联容器使用关键字的特点。

有序容器的迭代器通过关键字有序访问容器中的元素。无论在有序容器中还是在无序容器中，具有相同关键字的元素都是相邻存储的。

术语表

关联数组 (associative array) 元素通过关键字而不是位置来索引的数组。我们称这样的数组将一个关键字映射到其关联的值。

关联容器 (associative container) 类型，保存对象的集合，支持通过关键字的高效查找。

hash 特殊的标准库模板，无序容器用它来管理元素的位置。

哈希函数 (hash function) 将给定类型的值映射到整形 (`size_t`) 值的函数。相等的值必须映射到相同的整数；不相等的值应尽可能映射到不同整数。

key_type 关联容器定义的类型，用来保存和提取值的关键字的类型。对于一个 `map`，`key_type` 是用来索引 `map` 的类型。对于 `set`，`key_type` 和 `value_type` 是一样的。

map 关联容器类型，定义了一个关联数组。类似 `vector`，`map` 是一个类模板。但是，一个 `map` 要用两个类型来定义：关键字的类型和关联的值的类型。在一个 `map` 中，一个给定关键字只能出现一次。每个关键字关联一个特定的值。解引用一个 `map` 迭代器会生成一个 `pair`，它保存一个 `const` 关键字及其关联的值。

mapped_type 映射类型定义的类型，就是映射中关键字关联的值的类型。

multimap 关联容器类型，类似 `map`，不同之处在于，在一个 `multimap` 中，一个给定的关键字可以出现多次。`multimap` 不支持下标操作。

multiset 保存关键字的关联容器类型。在一个 `multiset` 中，一个给定关键字可以出现多次。

448